



norma

Guide d'installation et d'utilisation

Apprendre les contraintes de déni qui régissent une table, détecter les erreurs de qualité de données, et la normaliser.

Moteur : **CPAD** (Constrained Predicates for Anomaly Detection)

Apache-2.0 · github.com/normaCPAD/norma

Table des matières

1	Présentation	2
2	Installation	2
2.1	Depuis PyPI	2
2.2	Depuis les sources	2
2.3	Prérequis	2
3	Prise en main (ligne de commande)	2
4	Référence de la ligne de commande	2
4.1	<code>norma analyze</code>	2
4.2	<code>norma export</code> – vers votre pile qualité	3
4.3	<code>norma freeze</code> / <code>norma check</code> – contrats & CI	3
4.4	<code>norma score</code> – scorer toute la base	3
4.5	<code>norma bench</code> – évaluation reproductible	4
5	API Python & notebook	4
6	L’application de bureau (<code>norma studio</code>)	4
7	Construire un exécutable autonome	4
7.1	Icône de la barre des tâches Linux	5
8	Dépannage	5

1 Présentation

NORMA découvre les dépendances fonctionnelles (DF) et les contraintes de déni (DC) qu'une table relationnelle, supposée majoritairement propre, satisfait ; elle attribue à chaque cellule un score de *violation* (le moteur CPAD), puis en dérive les clés candidates, la forme normale courante et une décomposition 3NF/BCNF. Contrairement aux détecteurs d'anomalies classiques, elle distingue une **erreur relationnelle** (une valeur qui brise une contrainte) d'une valeur **rare mais valide**. Elle exporte enfin les contraintes découvertes vers les outils de qualité de données que vous utilisez déjà.

Trois modes d'utilisation : un outil en ligne de commande (`norma`), une bibliothèque Python (`import norma`) et une application de bureau interactive (`norma studio`).

2 Installation

2.1 Depuis PyPI

```
pip install norma-dq # noyau (le nom d'import reste : import norma)
pip install "norma-dq[gated]" # + PyTorch pour la variante différentiable GatedCPAD
pip install "norma-dq[studio]" # + PySide6 pour l'application de bureau
pip install "norma-dq[lake]" # + ingestion Parquet / DuckDB / Excel
pip install "norma-dq[baselines]" # + pyod (baseline Isolation Forest pour 'norma bench')
```

La distribution s'appelle `norma-dq` sur PyPI, mais le paquet Python que l'on importe est toujours `norma`.

2.2 Depuis les sources

```
git clone https://github.com/normaCPAD/norma.git
cd norma
pip install -e ".[studio,lake,baselines,dev]"
pytest -q # lance la suite de tests
```

2.3 Prérequis

Python ≥ 3.9 . Les dépendances du noyau (NumPy, pandas, SciPy, scikit-learn, PyYAML) sont installées automatiquement. La variante différentiable et l'application de bureau sont des extras optionnels.

3 Prise en main (ligne de commande)

À partir de n'importe quel fichier tabulaire, découvrez ses contraintes et un modèle de données proposé :

```
norma analyze data.csv
```

Cela affiche les contraintes découvertes (avec leur confiance), les clés candidates, la forme normale courante, une décomposition 3NF/BCNF proposée, et les cellules les plus en violation.

4 Référence de la ligne de commande

4.1 `norma analyze`

```
norma analyze CHEMIN [--learner routed|discrete|gated|ensemble]
                        [--tau 0.90] [--max-lhs 2] [--cap 300]
                        [--top 10] [--json model.json]
```

- **-learner** – apprenant ; **routed** (défaut) est le CPAD complet.
- **-tau** – confiance minimale d’une DF pour l’apprenant discret (**routed/gated** la sélectionnent seuls).
- **-max-lhs** – taille maximale du membre gauche pour les DF composées.
- **-cap** – cardinalité maximale d’une colonne considérée comme modélisable.
- **-top** – nombre de cellules en violation à afficher.
- **-json** – écrit aussi le modèle complet en JSON.

4.2 norma export – vers votre pile qualité

```
norma export CHEMIN --to FORMAT [--out DOSSIER] [--learner ...] [--tau ...]
```

-to	Sortie
great_expectations	Suite d’attentes (clés → unicité composée ; DF → requêtes)
pandera	DataFrameSchema avec contrôles DF par groupe et linéaires
dbt	schema.yml + un test singulier (SQL) par dépendance
sql_check	UNIQUE/CHECK ANSI + vues moniteurs de violations
json_schema	JSON Schema par enregistrement (types / requis)
frictionless	Table Schema (types + primaryKey/uniqueKeys)
metanome	DF/DC en syntaxe de résultats Metanome
shacl	formes SHACL avec sh:sparql (graphes de connaissances)

NORMA découvre les règles ; votre pipeline les applique.

4.3 norma freeze / norma check – contrats & CI

Gelez les contraintes acceptées dans un **norma.yml** versionné, qui vit dans git, puis revalidez n’importe quelle table contre lui en intégration continue :

```
norma freeze data.csv -o norma.yml
norma check data.csv -c norma.yml --fail-on-violations
```

check renvoie le code de sortie 1 dès qu’une contrainte est violée : il peut donc bloquer un pipeline directement. Une *clé candidate* découverte est vérifiée par la propriété robuste de *déterminant* (elle détermine toutes les colonnes), et non par l’unicité des lignes de la table à plat.

4.4 norma score – scorer toute la base

Applique les règles apprises à *chaque* ligne d’un fichier (même très volumineux), en le parcourant par morceaux pour borner la mémoire. Sans **-max-rows**, les dépendances fonctionnelles sont découvertes sur le fichier *entier* (streaming, sans échantillon) ; avec, elles sont apprises sur un échantillon (ce qui active aussi les contraintes composées et numériques) et toutes les lignes sont quand même scorées.

```
norma score data.csv # apprend les FD sur TOUT le fichier (streaming, sans echantillon), score
tout
norma score data.csv --out scored.csv # écrit aussi les colonnes norma_score / norma_rule
norma score data.csv --max-rows 200000 # apprend sur un echantillon (FD composees / numerique),
score tout
```

Avec `-out`, les données sont réécrites avec deux colonnes : `norma_score` (le degré de violation par ligne) et `norma_rule` (la contrainte responsable).

4.5 norma bench – évaluation reproductible

Évaluez la détection sur des paires propre/sale alignées (format Raha/Baran) et reportez les AUROC/AUPRC face à la vérité terrain, à côté d’une baseline Isolation Forest :

```
norma bench --data ./datasets --learner routed --json results.json
```

Les deux dispositions sont détectées automatiquement sous `-data` :

```
datasets/<nom>/clean.csv datasets/<nom>/dirty.csv  
datasets/<nom>_clean.csv datasets/<nom>_dirty.csv
```

5 API Python & notebook

```
from norma.core.table import Table  
from norma.models import RoutedCPAD  
from norma.modeling.report import build_model  
from norma.export import export  
  
t = Table.from_csv("data.csv") # ou from_parquet / from_excel / from_json / from_any  
report = build_model(t, RoutedCPAD().fit(t))  
print(report.to_text())  
  
files = export(report, "great_expectations") # {nom_fichier: contenu}
```

Dans Jupyter/Colab, `t.profile()` rend un rapport HTML en ligne dont les principales violations sont colorées : **violation de contrainte** contre **rare mais valide**.

```
from norma.core.table import Table  
Table.from_csv("data.csv").profile()
```

6 L’application de bureau (norma studio)

Lancez l’application interactive :

```
norma-studio # ou : python -m norma.studio
```

Fonctionnalités. Ouvrir un fichier CSV/Parquet/Excel/JSON ou se connecter à une base relationnelle ; découvrir des contraintes DF, composées, d’ordre et linéaires ; les activer/désactiver et ajouter des règles expertes (le schéma se re-synthétise en direct) ; diagrammes interactifs du graphe de DF et du schéma relationnel ; visuel ↔ SQL bidirectionnel ; une carte de chaleur des anomalies ; réparation guidée par les contraintes ; un rapport qualité en un clic ; et un constructeur de base propre. Le menu **Fichier** exporte aussi les contraintes vers n’importe quel format d’écosystème, et gèle/vérifie un contrat `norma.yml`. Bilingue (FR/EN), thèmes clair/sombre.

7 Construire un exécutable autonome

Un paquet de bureau autonome est produit avec PyInstaller :

```
cd norma  
bash packaging/build_linux.sh # -> dist/norma-studio/norma-studio
```

Le script encode trois correctifs d'environnement ; utilisez-le (plutôt que d'invoquer `pyinstaller` directement) pour qu'ils s'appliquent tous :

1. **Limite de récursion** – le graphe d'imports profond de NumPy/SciPy/ scikit-learn dépasse la valeur par défaut de Python ; le spec la relève (`sys.setrecursionlimit`).
2. **Conflit de bibliothèques Kerberos** (conda vs. système) – le `QtNetwork` de Qt est importé pendant l'analyse ; le script précharge une famille `libkrb5` cohérente.
3. **Liens symboliques sur exFAT/NTFS** – PyInstaller lie les bibliothèques partagées ; le script construit sur un système de fichiers natif puis recopie le paquet sans liens (`rsync -L`), pour qu'il fonctionne sur disque externe.

Windows / macOS. Le spec embarque `icon.ico` (Windows) et `icon.icns` (macOS, avec un paquet `.app`). Construisez sur l'OS cible avec `pyinstaller packaging/norma-studio.spec`.

7.1 Icône de la barre des tâches Linux

L'icône de la fenêtre est définie dans l'application ; l'icône de la *barre des tâches* provient d'une entrée de bureau. Installez-la une fois :

```
bash packaging/install_linux_desktop.sh
```

Pour un build packagé, pointez le lanceur vers le binaire embarqué :

```
sed -i "s|^Exec=.*|Exec=$PWD/dist/norma-studio/norma-studio|" \
~/.local/share/applications/norma-studio.desktop
```

8 Dépannage

Spec file ... not found

Vous êtes dans le mauvais dossier. Lancez depuis la racine du projet `norma`, là où se trouve `packaging/`.

RecursionError pendant un build

Utilisez `bash packaging/build_linux.sh` ; si cela persiste, augmentez `sys.setrecursionlimit` dans le spec.

undefined symbol: k5_buf_cstring

Conflit Kerberos conda/système quand Qt charge `QtNetwork`. Construisez via le script, ou préfixez la commande par `LD_LIBRARY_PATH=$CONDA_PREFIX/lib`.

os.symlink ... Operation not permitted

Le dossier de sortie est sur exFAT/NTFS. Construisez via le script (il recopie sans liens), ou visez un système de fichiers natif.

La barre des tâches montre une icône générique

Lancez `install_linux_desktop.sh` et démarrez l'application via la commande `norma-studio` pour que sa classe de fenêtre corresponde à l'entrée de bureau.